

This is unreleased documentation for Serverpod **Next** version.

For up-to-date documentation, see the [latest version](#) (4.0.0).

3..13

ENDPOINT INHERITANCE

 Copy as Markdown

Endpoint inheritance

An endpoint can extend another endpoint, declared like any Dart class: `class ChildEndpoint extends ParentEndpoint`. Use inheritance to share behavior across endpoints, such as [auth requirements](#), to extend or reshape endpoints from modules, and to control what the generated client exposes.

How the parent is marked decides what gets generated for it. An `abstract` parent is not exposed on the server, and a `@doNotGenerate` parent gets no client code. Your concrete subclass always gets a client, unless it opts out itself. The sections below walk through each case.

Inheriting from an Endpoint class

Given an existing `Endpoint` class, it is possible to extend or modify its behavior while retaining the already exposed methods.

```
import 'package:serverpod/serverpod.dart';

class CalculatorEndpoint extends Endpoint {
  Future<int> add(Session session, int a, int b) async {
    return a + b;
  }
}

class MyCalculatorEndpoint extends CalculatorEndpoint {
  Future<int> subtract(Session session, int a, int b) async {
    return a - b;
  }
}
```

The generated client code will now be able to access both `CalculatorEndpoint` and `MyCalculatorEndpoint`. Whereas the `CalculatorEndpoint` only exposes the original `add` method, `MyCalculatorEndpoint` now exposes both the inherited `add` and its own `subtract` method.



Ask AI

Inheriting from an `Endpoint` class marked `abstract`

Endpoints marked as `abstract` are not added to the server. But if they are subclassed, their methods will be exposed through the subclass.

```
import 'package:serverpod/serverpod.dart';

abstract class CalculatorEndpoint extends Endpoint {
  Future<int> add(Session session, int a, int b) async {
    return a + b;
  }
}

class MyCalculatorEndpoint extends CalculatorEndpoint {}
```

Since `CalculatorEndpoint` is `abstract`, it will not be exposed on the server. However, an abstract client class will be generated, which will be extended by the class generated from `MyCalculatorEndpoint`. The concrete client exposes the `add` method it inherited from `CalculatorEndpoint`. See [Client-side endpoint inheritance](#) for more details on how abstract endpoints are represented on the client.

Extending an `abstract Endpoint` class

In the above example, the `MyCalculatorEndpoint` only exposed the inherited `add` method. It can be further extended with custom methods like this:

```
import 'package:serverpod/serverpod.dart';

class MyCalculatorEndpoint extends CalculatorEndpoint {
  Future<int> subtract(Session session, int a, int b) async {
    return a - b;
  }
}
```

In this case, it will expose both an `add` and a `subtract` method.

Inheriting from an `Endpoint` class annotated with `@doNotGenerate`

Suppose you had an `Endpoint` class marked with `@doNotGenerate` and a subclass that extends it:

```
import 'package:serverpod/serverpod.dart';

@doNotGenerate
class CalculatorEndpoint extends Endpoint {
  Future<int> add(Session session, int a, int b) async {
    return a + b;
  }
}
```

```
class MyCalculatorEndpoint extends CalculatorEndpoint {}
```

Since `CalculatorEndpoint` is marked as `@doNotGenerate`, it will not be exposed on the server, and no client class will be generated for it. Only `MyCalculatorEndpoint` will be accessible from the client, which provides the inherited `add` method from its parent class. When a parent is marked with `@doNotGenerate`, no client class is generated for the parent. The subclass's client class extends `EndpointRef` directly, with the parent's methods inlined alongside the subclass's own methods.

Overriding endpoint methods

It is possible to override methods of the superclass. This can be useful when you want to modify the behavior of specific methods but preserve the rest.

```
import 'package:serverpod/serverpod.dart';

abstract class GreeterBaseEndpoint extends Endpoint {
  Future<String> greet(Session session, String name) async {
    return 'Hello $name';
  }
}

class ExcitedGreeterEndpoint extends GreeterBaseEndpoint {
  @override
  Future<String> greet(Session session, String name) async {
    return '${await super.greet(session, name)}!!!';
  }
}
```

Since `GreeterBaseEndpoint` is `abstract`, it will not be exposed on the server. The `ExcitedGreeterEndpoint` will expose a single `greet` method, and its implementation will augment the superclass's one by adding `!!!` to that result.

This way, you can modify the behavior of endpoint methods while still sharing the implementation through calls to `super`. Be aware that the method signature has to be compatible with the base class per Dart's rules, meaning you can add optional parameters, but can not add required parameters or change the return type.

Hiding endpoint methods with `@doNotGenerate`

In case you want to hide a method that the child class declares itself (i.e. one that is not inherited from a parent class whose client is still being generated), annotate it with `@doNotGenerate` like so:

```
import 'package:serverpod/serverpod.dart';

class AdderEndpoint extends Endpoint {
  Future<int> add(Session session, int a, int b) async {
    return a + b;
  }
}
```

```

    }

    @doNotGenerate
    Future<int> subtract(Session session, int a, int b) async {
        throw UnimplementedError();
    }
}

```

The `AdderEndpoint` exposes `add`, but `subtract` is annotated with `@doNotGenerate` and is therefore excluded from the generated client.

Hiding an inherited method

Hiding a method that is *inherited* from a parent class only works if the parent class itself is fully annotated with `@doNotGenerate`, meaning no client class is generated for it at all:

```

import 'package:serverpod/serverpod.dart';

@doNotGenerate
class CalculatorEndpoint extends Endpoint {
    Future<int> add(Session session, int a, int b) async {
        return a + b;
    }

    Future<int> subtract(Session session, int a, int b) async {
        return a - b;
    }
}

class AdderEndpoint extends CalculatorEndpoint {
    @doNotGenerate
    @override
    Future<int> subtract(Session session, int a, int b) async {
        throw UnimplementedError();
    }
}

```

Since `CalculatorEndpoint` is annotated with `@doNotGenerate`, it will not be exposed on the server, and no client class is generated for it. `AdderEndpoint` inlines the inherited methods directly, and since it re-declares `subtract` with `@doNotGenerate`, only `add` is exposed on the generated client. Don't worry about the exception in the `subtract` implementation. That is only added to satisfy the Dart compiler. In practice, nothing will ever call this method on `AdderEndpoint`.

⚠ WARNING

If the parent class is only `abstract` (and not itself annotated with `@doNotGenerate`), Serverpod still generates an abstract client class that mirrors it, and every subclass's generated client must implement all of its methods. In that case, `@doNotGenerate` **cannot** be used to hide an inherited method. Doing so removes the method from the subclass's generated client while the abstract client class still declares it,

which causes a Dart compile error ("missing concrete implementation"). To hide an inherited method, the parent class must be marked `@doNotGenerate` itself, not just `abstract`.

If the parent class is a normal, concrete class (neither `abstract` nor `@doNotGenerate`), it is exposed on the server and the client in its own right. Hiding the method on the child only removes it from the child's client. It remains accessible through the parent's own generated client class.

Unhiding endpoint methods annotated with `@doNotGenerate` in the super class

The reverse of the previous example would be a base endpoint that has a method marked with `@doNotGenerate`, which you now want to expose on the subclass.

```
import 'package:serverpod/serverpod.dart';

abstract class CalculatorEndpoint extends Endpoint {
  Future<int> add(Session session, int a, int b) async {
    return a + b;
  }

  // Ignored, as this expensive computation should not be exposed by default
  @doNotGenerate
  Future<BigInt> addBig(Session session, BigInt a, BigInt b) async {
    return a + b;
  }
}

class MyCalculatorEndpoint extends CalculatorEndpoint {
  @override
  Future<BigInt> addBig(Session session, BigInt a, BigInt b) async {
    return super.addBig(session, a, b);
  }
}
```

Since `CalculatorEndpoint` is `abstract`, it will not be exposed on the server. The generated `MyCalculatorEndpoint` client will expose both the `add` and `addBig` methods, since annotations are not inherited: overriding `addBig` without repeating `@doNotGenerate` re-exposes it.

Building base endpoints for behavior

Beyond inheriting or hiding methods, endpoint subclassing can pre-configure any other property of the `Endpoint` class.

For example, you could define a base class that requires callers to be signed in, using the `requireLogin` flag from [authentication on endpoints](#):

```
abstract class LoggedInEndpoint extends Endpoint {
```

```
@override
bool get requireLogin => true;
}
```

And now every endpoint that extends `LoggedInEndpoint` will check that the user is logged in.

Similarly, you could wrap up a specific set of required scopes in a base endpoint, and use it for the app's endpoints instead of repeating the scopes in each:

```
abstract class AdminEndpoint extends Endpoint {
  @override
  Set<Scope> get requiredScopes => {Scope.admin};
}
```

Have your custom endpoint extend `AdminEndpoint`, and the user is guaranteed to hold the appropriate scopes.

Client-side endpoint inheritance

When you use endpoint inheritance on the server, Serverpod generates matching client-side classes that mirror your inheritance hierarchy. This allows you to write type-safe client code that works with abstract endpoint types.

Abstract endpoint client generation

When you define an abstract endpoint on the server, Serverpod generates an abstract client endpoint class. This is particularly useful for module developers who want to provide base functionality that users can extend.

Server-side abstract endpoint:

```
import 'package:serverpod/serverpod.dart';

abstract class CalculatorEndpoint extends Endpoint {
  Future<int> add(Session session, int a, int b) async {
    return a + b;
  }
}
```

Generated client-side abstract class:

```
abstract class EndpointCalculator extends EndpointRef {
  EndpointCalculator(EndpointCaller caller) : super(caller);

  Future<int> add(int a, int b);
}
```

When you extend this abstract endpoint in your server:

```
class MyCalculatorEndpoint extends CalculatorEndpoint {
  Future<int> subtract(Session session, int a, int b) async {
    return a - b;
  }
}
```

The generated client class will extend the abstract client class:

```
class EndpointMyCalculator extends EndpointCalculator {
  EndpointMyCalculator(EndpointCaller caller) : super(caller);

  @override
  String get name => 'myCalculator';

  @override
  Future<int> add(int a, int b) => caller.callServerEndpoint<int>(
    'myCalculator',
    'add',
    {'a': a, 'b': b},
  );

  Future<int> subtract(int a, int b) => caller.callServerEndpoint<int>(
    'myCalculator',
    'subtract',
    {'a': a, 'b': b},
  );
}
```

Using `getEndpointOfType` for type-safe endpoint access

When working with abstract endpoints, you can use the `getEndpointOfType` method to retrieve concrete endpoint instances by their type. This is especially useful when writing code that depends on abstract endpoint interfaces provided by modules.

```
// Get an endpoint by its type
var calculator = client.getEndpointOfType<EndpointCalculator>();

// Now you can call methods defined in the abstract base class
var result = await calculator.add(5, 3);
```

The `getEndpointOfType` method will:

- Return the single endpoint of the requested type if only one exists.
- Throw `ServerpodClientEndpointNotFound` if no endpoint of that type is found.
- Throw `ServerpodClientMultipleEndpointsFound` if multiple endpoints of that type exist.

Disambiguating multiple endpoints

If you have multiple concrete implementations of the same abstract endpoint, you can disambiguate by providing the endpoint name:

```
// Server-side: Two implementations of the same abstract endpoint
class BasicCalculatorEndpoint extends CalculatorEndpoint {}
class AdvancedCalculatorEndpoint extends CalculatorEndpoint {
  Future<int> multiply(Session session, int a, int b) async {
    return a * b;
  }
}
```

```
// Client-side: Specify which implementation you want
var basicCalc = client.getEndpointOfType<EndpointCalculator>('basicCalculator');
var advancedCalc = client.getEndpointOfType<EndpointCalculator>('advancedCalculator');
```

Use case: Module-provided abstract endpoints

This pattern is especially useful for modules. A module can provide an abstract endpoint that defines an interface, and users of the module can extend it to expose the functionality on their server:

In a module:

Declare an abstract endpoint with common methods on the server:

```
abstract class AuthSessionEndpoint extends Endpoint {
  Future<bool> isAuthenticated(Session session) async {
    return session.isUserSignedIn;
  }

  Future<bool> logout(Session session, {required bool allSessions}) async {
    // Implementation...
  }
}
```

Write client-side code that depends on the generated abstract type and uses its methods:

```
class UserLoggedInWidget extends StatelessWidget {
  final Client client;

  UserLoggedInWidget({required this.client});

  // Will throw if no concrete implementation of the endpoint exists.
  EndpointAuthSession get sessionEndpoint =>
    client.getEndpointOfType<EndpointAuthSession>();
}
```

```

@override
Widget build(BuildContext context) {
  return FutureBuilder<bool>({
    future: sessionEndpoint.isAuthenticated(),
    builder: (context, snapshot) {
      if (snapshot.connectionState == ConnectionState.waiting) {
        return CircularProgressIndicator();
      } else if (snapshot.hasError) {
        return Text('Error: ${snapshot.error}');
      } else if (snapshot.hasData && snapshot.data == true) {
        return Text('User is logged in');
      } else {
        return Text('User is not logged in');
      }
    },
  );
}

```

In the user application:

The user extends the abstract endpoint to expose it on their server. Then, any client code that depends on the abstract endpoint will work regardless of the concrete class name or location.

```

// Extend the module's abstract endpoint to expose it
class SessionEndpoint extends AuthSessionEndpoint {}

```

This approach allows module developers to provide reusable endpoint logic while giving application developers full control over which endpoints are exposed on their server.